Practical No: 1

Implement A* Search for 8 puzzle problem

Title
Implementation of A Search Algorithm for Solving the 8-Puzzle Problem.

Objectives
- To understand the working of the A* Search Algorithm.
- To solve the 8-Puzzle problem using heuristic search.
- To learn how informed search algorithms find the shortest path.
- To compare the initial state with the goal state.
- To understand heuristic functions in Artificial Intelligence.

Problem Statement
Implement the *A Search Algorithm** to solve the **8-Puzzle Problem**. The program should find the shortest sequence of moves from the given initial state to the goal state using the heuristic function.

Software Requirements
- Python 3.10 or above
- Visual Studio Code (VS Code) / PyCharm / Jupyter Notebook
- Windows/Linux/MacOS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Hard Disk: 500 MB free space
- Keyboard and Mouse

Theory
What is an 8-Puzzle Problem?
The **8-Puzzle** is a classic Artificial Intelligence problem.

It consists of
Eight numbered tiles (1 to 8)
One empty space (represented by 0 or blank)

Example
Initial State
1 2 3
4 0 6
7 5 8

Goal State
1 2 3
4 5 6
7 8 0
The objective is to move the blank space so that all tiles reach the goal position.

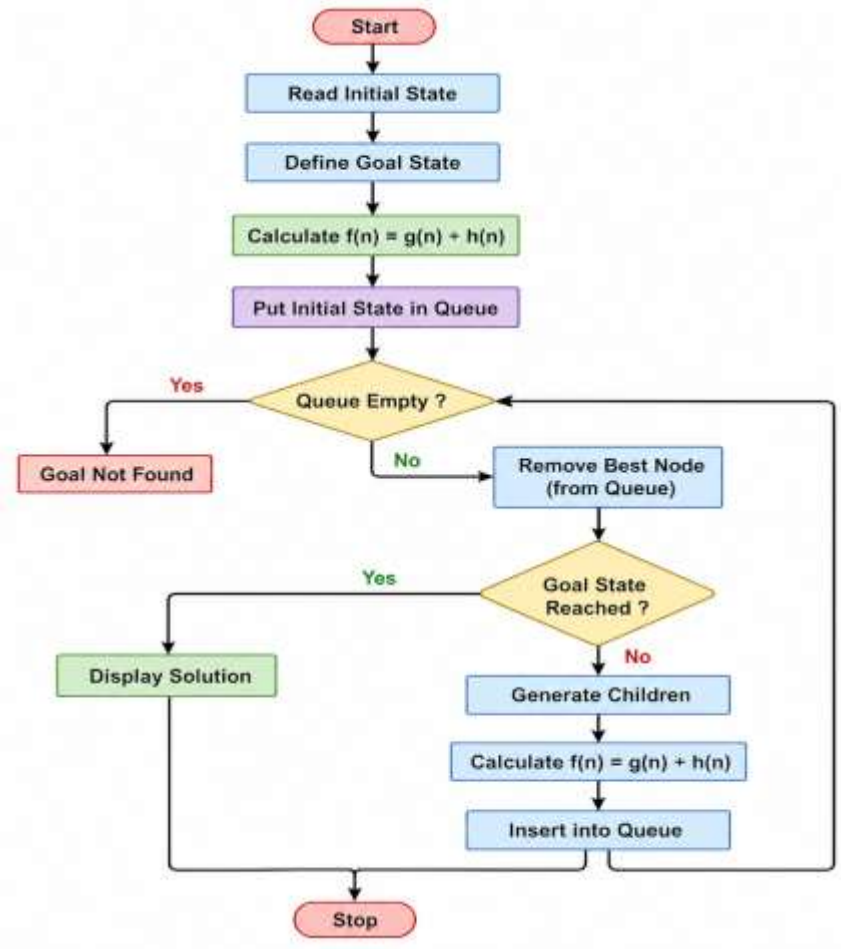
What is A* Search?
A* (A-Star) is an informed search algorithm.
It finds the shortest path by evaluating every state using
$f(n) = g(n) + h(n)$
Where
g(n) = Cost from Start Node to Current Node
h(n) = Estimated Cost from Current Node to Goal Node
f(n) = Total Estimated Cost
The node with the smallest **f(n)** value is expanded first.

Heuristic Function
For the 8-Puzzle, the most common heuristic is
Manhattan Distance
It is the total distance each tile is away from its correct position.
Formula
$h(n)=\sum|x1-x2|+|y1-y2|$
The smaller the heuristic value, the closer the puzzle is to the goal state.

Algorithm

1. Read the initial state.
2. Define the goal state.
3. Calculate heuristic value using Manhattan Distance.
4. Calculate
    - g(n)
    - h(n)
    - f(n)=g(n)+h(n)
5. Store states in a priority queue.
6. Select the state having the lowest f(n).
7. Generate all possible child states.
8. Ignore already visited states.
9. Continue until the goal state is found.
10. Display the solution path

Flowchart



Mathematical Model

Let

- **S** = Set of all puzzle states
- **A** = Set of possible actions (Up, Down, Left, Right)
- $\mathbf{s_0}$ = Initial State
- **G** = Goal State
- **g(n)** = Cost from start node
- **h(n)** = Manhattan Distance heuristic
- **f(n)** = g(n) + h(n)

State Space Representation

S=(s0,A,G)

Evaluation Function
$f(n)=g(n)+h(n)$

Heuristic Function
$h(n)=\sum|x1-x2|+|y1-y2|$

Conclusion

The *A Search Algorithm** efficiently solves the **8-Puzzle Problem** by combining the actual path cost (**g(n)**) with the heuristic estimate (**h(n)**). Using the **Manhattan Distance** heuristic, A* explores the most promising states first and finds the shortest path to the goal. This practical demonstrates the use of informed search techniques in Artificial Intelligence and their effectiveness in solving state-space search problems.

Practical Questions

1. Implement the A* Search Algorithm to solve the 8-Puzzle Problem using the Manhattan Distance heuristic.
2. Write a Python program to find the shortest path from the initial state to the goal state using the A* Search Algorithms.
3. Implement the A* Search Algorithm and display the sequence of moves required to reach the goal state.
4. Calculate the evaluation function $f(n)=g(n)+h(n)$f(n) = g(n) + h(n)$f(n)=g(n)+h(n)$ for each state while solving the 8-Puzzle Problem using A*.
5. Compare the performance of the A* Search Algorithm with Breadth-First Search (BFS) for solving the 8-Puzzle Problem in terms of path cost and nodes explored.

Code:

```
from queue import PriorityQueue

goal = ((1,2,3),
        (4,5,6),
        (7,8,0))

goal_pos = {}
for i in range(3):
    for j in range(3):
        goal_pos[goal[i][j]] = (i, j)

def heuristic(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
```

```
                x, y = goal_pos[value]
                distance += abs(i - x) + abs(j - y)
    return distance

def get_neighbors(state):
    neighbors = []
    state = [list(row) for row in state]

    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    moves = [(-1,0),(1,0),(0,-1),(0,1)]

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def astar(start):

    pq = PriorityQueue()
    pq.put((heuristic(start), 0, start))

    visited = set()

    while not pq.empty():

        f, g, current = pq.get()

        if current == goal:
            print("Goal State Reached")
            return

        visited.add(current)

        for neighbor in get_neighbors(current):
            if neighbor not in visited:
```

```
            h = heuristic(neighbor)
            pq.put((g+1+h, g+1, neighbor))

start = ((1,2,3),
        (4,0,6),
        (7,5,8))

astar(start)
```

Practical No: 2

Implementation of Adversarial Search with Alpha-Beta Pruning

Title

Implementation of Adversarial Search with Alpha-Beta Pruning

Objectives

- To understand the concept of adversarial search used in Artificial Intelligence.
- To implement the Minimax algorithm with Alpha-Beta pruning.
- To reduce the number of nodes evaluated by pruning unnecessary branches.
- To find the optimal decision for a player in a competitive environment.
- To analyze the performance improvement achieved using Alpha-Beta pruning.

Problem Statement

Implement an **Adversarial Search algorithm using Alpha-Beta pruning** for a two-player game environment. The algorithm should determine the best possible move for the maximizing player by eliminating unnecessary searches in the game tree.

Example: Implement Alpha-Beta pruning for a simple game tree where:

- MAX player tries to maximize the score.
- MIN player tries to minimize the score.
- Alpha (α) represents the best value found for MAX.
- Beta (β) represents the best value found for MIN.

Software Requirements:

Programming Language: Python 3.x
Platform/IDE:

- Visual Studio Code
- PyCharm
- Jupyter Notebook (Optional)

Operating System: Windows/Linux/MacOS

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Adversarial Search

Adversarial search is an AI search technique used in competitive environments where one player's gain is another player's loss.

Examples:

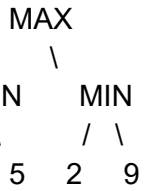
- Chess
- Tic-Tac-Toe
- Checkers
- Strategy games

In adversarial search:

- **MAX Player:** Attempts to maximize the score.
- **MIN Player:** Attempts to minimize the score.

Minimax Algorithm
Minimax is a decision-making algorithm that explores all possible moves and selects the best move assuming that the opponent also plays optimally.

Example:



MAX selects the maximum value from MIN decisions.

Alpha-Beta Pruning
Alpha-Beta pruning improves Minimax by removing branches that do not affect the final decision.

Parameters:

- Alpha (α)

Maximum value that MAX can guarantee.
Initial value = -∞

- Beta (β)

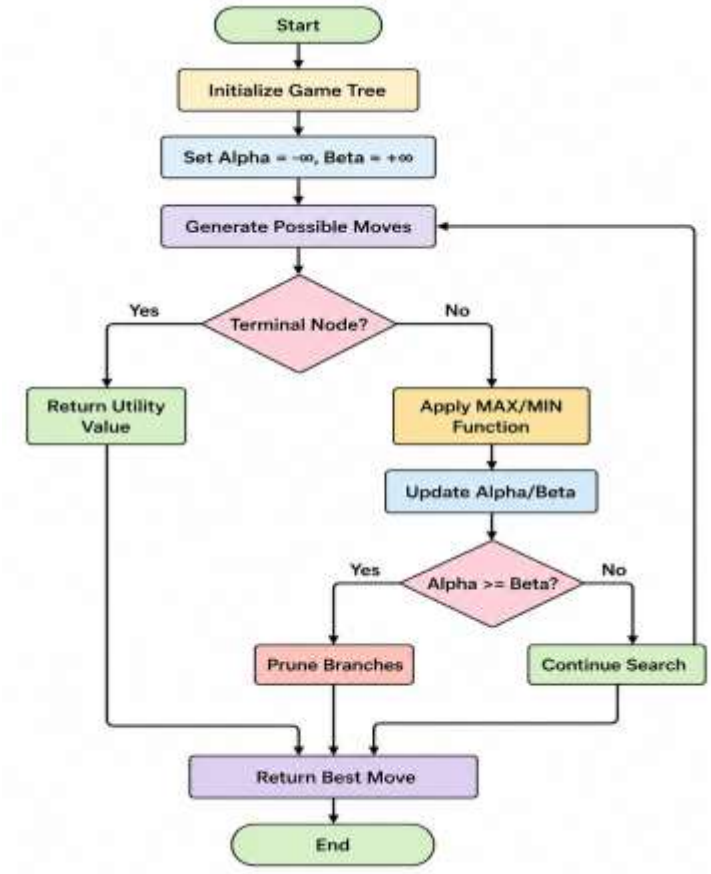
Minimum value that MIN can guarantee.
Initial value = +∞

Pruning Condition:
If Alpha >= Beta
Remaining branches are ignored
This reduces computation time without changing the final result.

Algorithm

1. Start the algorithm.
2. Create the game tree with possible moves.
3. Initialize:
   Alpha = -∞
   Beta = +∞
4. Check whether the current node is a terminal node.
5. If terminal node, return the utility value.
6. For MAX node:
   Select maximum value.
   Update Alpha.
7. For MIN node:
   Select minimum value.
   Update Beta.
8. If Alpha ≥ Beta, prune remaining branches
9. Return the best move.
10. Stop.

Flowchart



Mathematical Model

For Minimax:
MAX Function
MAX(n)=max(V(n1),V(n2),...,V(nk))
MIN Function
MIN(n)=min(V(n1),V(n2),...,V(nk))
Alpha-Beta Pruning:
Pruneif$\alpha \geq \beta$

Time Complexity:
Without pruning:
O(bd)
With Alpha-Beta pruning:
O(bd/2)

Where:

- **b = branching factor**
- **d = depth of tree**

Conclusion

The Adversarial Search algorithm with Alpha-Beta pruning was successfully implemented using Python. The algorithm efficiently finds the optimal decision by eliminating unnecessary branches from the search tree. Alpha-Beta pruning improves the performance of the Minimax algorithm by reducing the number of nodes evaluated while producing the same optimal result.

Practical Questions

1. Implement the Minimax algorithm for a Tic-Tac-Toe game.
2. Compare the performance of Minimax and Alpha-Beta pruning algorithms.
3. Implement Alpha-Beta pruning for a Chess move decision system.
4. Generate a game tree and demonstrate branch pruning using Alpha-Beta values.
5. Modify the Alpha-Beta algorithm to find the best move in a two-player board game.

Code:

```
def alpha_beta(depth, node_index, maximizing_player, values, alpha, beta):

    if depth == 3:
        return values[node_index]

    if maximizing_player:
```

```
        best = -9999

        for i in range(2):
            value = alpha_beta(depth + 1,
                               node_index * 2 + i,
                               False,
                               values,
                               alpha,
                               beta)

            best = max(best, value)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = 9999

        for i in range(2):
            value = alpha_beta(depth + 1,
                               node_index * 2 + i,
                               True,
                               values,
                               alpha,
                               beta)

            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
                break

        return best

values = [3, 5, 6, 9, 1, 2, 0, -1]

result = alpha_beta(0, 0, True, values, -9999, 9999)

print("Optimal Value:", result)
```

Practical No: 3

Implement Tower of Hanoi by State Space Search

Title

Implementation of Tower of Hanoi using State Space Search

Objectives

- To understand the concept of **State Space Search** in Artificial Intelligence.
- To represent the Tower of Hanoi problem as a state-space problem.
- To implement the search technique to find the sequence of moves required to solve the problem.
- To understand state representation and transition between states.
- To analyze the solution path from initial state to goal state.

Problem Statement

Implement the **Tower of Hanoi problem using State Space Search**.
The problem consists of three rods:

- **Source Rod (A)**
- **Auxiliary Rod (B)**
- **Destination Rod (C)**

The objective is to move all disks from the source rod to the destination rod following these rules:

- Only one disk can be moved at a time.
- A larger disk cannot be placed on a smaller disk.
- All disks must be transferred from the source rod to the destination rod.

The problem should be solved by representing different configurations as states and searching for the goal state.

Software Requirements:

Programming Language: Python 3.x

Platform / IDE:

- Visual Studio Code
- PyCharm
- Jupyter Notebook

Operating System:

- Windows / Linux / macOS

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space

Theory

State Space Search

State Space Search is an AI problem-solving technique where a problem is represented using:

- **Initial State**
- **Operators / Actions**
- **Goal State**
- **Path Cost**

The algorithm searches through possible states to reach the desired goal state.

Tower of Hanoi as State Space Problem

Initial State:

All disks are placed on Source Rod A.

Example:

A = [3,2,1]

B = []

C = []

Goal State:

All disks are moved to Destination Rod C.

A = []

B = []

C = [3,2,1]

State Transition:

Each valid disk movement creates a new state.

Example:

Move disk 1 from A to C

A = [3,2]

B = []

C = [1]

State Space Representation

A state can be represented as:

S=(A,B,C)

Where:

- A = Source rod configuration
- B = Auxiliary rod configuration
- C = Destination rod configuration

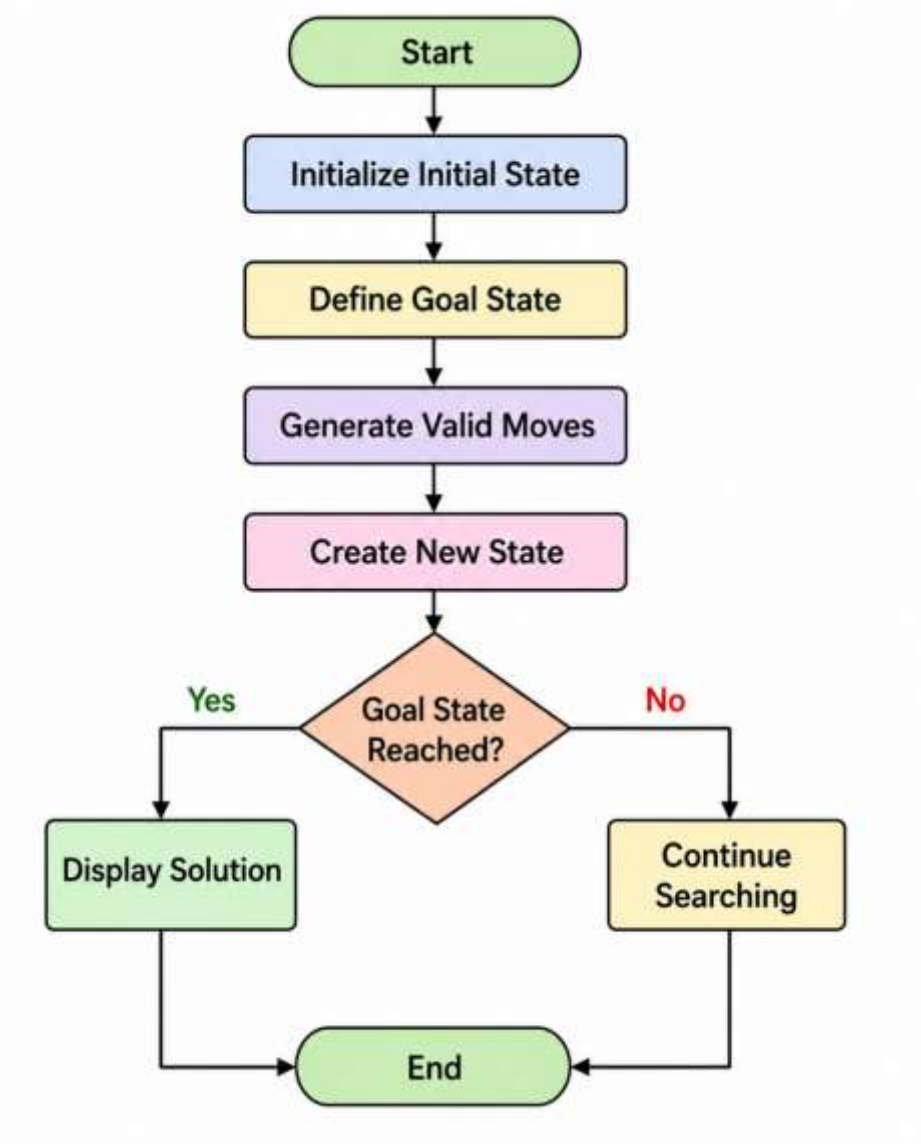
Algorithm

1. Start the program.
2. Define the initial state:

(Source, Auxiliary, Destination)

3. Generate possible valid moves.
4. Create new states after every valid move.
5. Check whether the current state matches the goal state.
6. If goal state is reached, display the solution path.
7. Otherwise continue searching the state space.
8. Stop.

Flowchart



Mathematical Model

For Tower of Hanoi:
Number of minimum moves:
$T(n)=2n-1$
Where:

- **n = Number of disks**
- **T(n) = Total number of moves**

Examples:
For n = 3:
$T(3)=23-1$
$T(3)=7$

State Space Model:
$S=(A,B,C)$
Initial State:
$S0=([n,n-1,...,1],[],[])$
Goal State:
$Sg=([],[],[n,n-1,...,1])$

Conclusion
The Tower of Hanoi problem was successfully implemented using State Space Search. The problem was represented using states and transitions between states. The algorithm successfully finds the sequence of moves required to reach the goal state while maintaining the given constraints.

Practical Questions

1. Implement the Water Jug problem using State Space Search.
2. Implement the 8-Puzzle problem using Breadth First Search.
3. Solve the Missionaries and Cannibals problem using State Space Representation.
4. Implement the Vacuum Cleaner problem using State Space Search.
5. Compare Depth First Search and Breadth First Search techniques for solving state space problems

Code:

```
def tower_of_hanoi(n, source, auxiliary, destination, moves):

    if n == 1:
        moves.append(
            f"Move disk 1 from {source} to {destination}"
        )
        return
```

```
    tower_of_hanoi(n-1, source, destination, auxiliary, moves)

    moves.append(
        f"Move disk {n} from {source} to {destination}"
    )

    tower_of_hanoi(n-1, auxiliary, source, destination, moves)


n = 3

moves = []

tower_of_hanoi(n, 'A', 'B', 'C', moves)

print("Solution Path:")

for move in moves:
    print(move)

print("Total Moves:", len(moves))
```

Practical No: 4

To implement a Sudoku solver using backtracking and constraint propagation techniques.

Title:

Implementation of Sudoku Solver using Backtracking and Constraint Propagation Techniques

Objectives

To understand the concept of **Constraint Satisfaction Problems (CSP)** in Artificial Intelligence.
To implement a Sudoku solver using the **Backtracking algorithm**.
To apply **Constraint Propagation** techniques to reduce the search space.
To find a valid solution for a given Sudoku puzzle.
To analyze the efficiency of AI-based problem-solving techniques.

Problem Statement

Implement a **Sudoku Solver using Backtracking and Constraint Propagation techniques**.

Given a partially filled **9×9 Sudoku grid**, the program should fill the empty cells according to Sudoku rules:

Rules:

1. Each row must contain numbers from 1 to 9 without repetition.
2. Each column must contain numbers from 1 to 9 without repetition.
3. Each 3×3 sub-grid must contain numbers from 1 to 9 without repetition.

The algorithm should use:

- **Backtracking** for searching possible solutions.
- **Constraint Propagation** for reducing possible values before searching.

Software Requirements:

Programming Language: Python 3.x

Platform / IDE:

- Visual Studio Code
- PyCharm
- Jupyter Notebook

Operating System:

- Windows / Linux / macOS

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Sudoku as Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem consists of:

1. **Variables**
2. **Domains**
3. **Constraints**

For Sudoku:

Variables:

Empty cells of the Sudoku grid.

Example:

Cell (1,3), Cell (4,5)

Domain:

Possible values that can be placed in each empty cell. \

Example:

Cell (1,3) = {2,4,7}

Constraints:

Rules that restrict possible values:

- Row constraint
- Column constraint
- 3×3 block constraint

Backtracking Technique

Backtracking is a depth-first search method that:

1. Selects an empty cell.
2. Tries possible values.
3. Checks constraints.
4. If valid, continues searching.
5. If conflict occurs, it goes back and tries another value.

Constraint Propagation

Constraint propagation reduces the search space by eliminating impossible values before performing search.

Example:

If a row already contains:

1 2 3 4

Then remaining cells cannot contain:

1,2,3,4

Algorithm

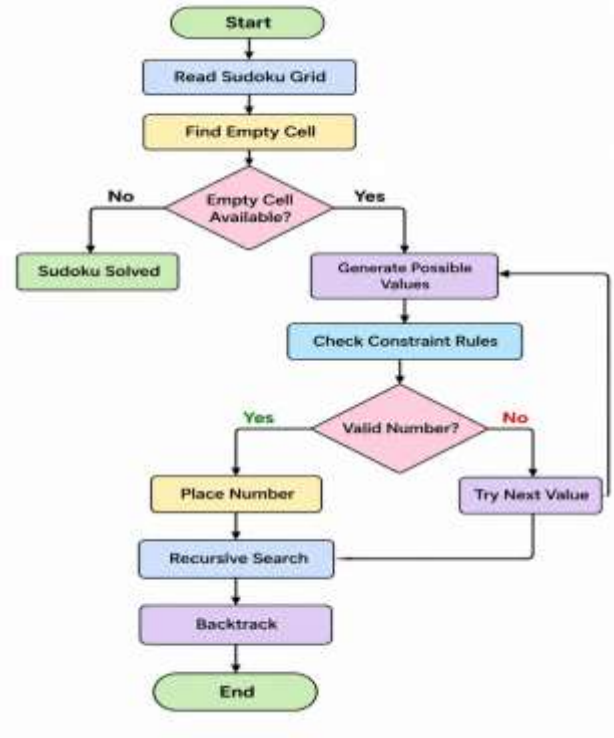
1. Start the program.
2. Read the Sudoku puzzle.
3. Find an empty cell.
4. Generate possible values using constraints.

5. Select one possible value.
6. Check whether the value satisfies:

Row constraint
Column constraint
Sub-grid constraint

7. If valid, place the number and continue recursively.
8. If no value works, backtrack to the previous cell.
9. Repeat until Sudoku is completely solved.
10. Display the solution.

Flowchart



Conclusion

The Sudoku Solver was successfully implemented using Backtracking and Constraint Propagation techniques. The algorithm efficiently searches for the solution by reducing unnecessary possibilities through constraints. This demonstrates the application of Artificial Intelligence techniques for solving Constraint Satisfaction Problems.

Practical Questions

1. Implement the N-Queens problem using Backtracking technique.
2. Implement Map Coloring problem using Constraint Satisfaction Problem.
3. Develop a Cryptarithmetic puzzle solver using Constraint Propagation.
4. Implement Graph Coloring using Backtracking algorithm.

5. Compare simple Backtracking and Constraint Propagation approaches for solving Sudoku.

Code:

```
def print_board(board):
    for row in board:
        print(row)


def is_safe(board, row, col, num):

    for i in range(9):
        if board[row][i] == num:
            return False

    for i in range(9):
        if board[i][col] == num:
            return False

    start_row = row - row % 3
    start_col = col - col % 3

    for i in range(3):
        for j in range(3):
            if board[start_row+i][start_col+j] == num:
                return False

    return True


def solve_sudoku(board):

    for row in range(9):
        for col in range(9):

            if board[row][col] == 0:

                for num in range(1,10):

                    if is_safe(board,row,col,num):

                        board[row][col] = num
```

```
                    if solve_sudoku(board):
                        return True

                    board[row][col] = 0

                return False

    return True


sudoku = [

[5,3,0,0,7,0,0,0,0],
[6,0,0,1,9,5,0,0,0],
[0,9,8,0,0,0,0,6,0],
[8,0,0,0,6,0,0,0,3],
[4,0,0,8,0,3,0,0,1],
[7,0,0,0,2,0,0,0,6],
[0,6,0,0,0,0,2,8,0],
[0,0,0,4,1,9,0,0,5],
[0,0,0,0,8,0,0,7,9]

]


if solve_sudoku(sudoku):
    print("Solved Sudoku:")
    print_board(sudoku)

else:
    print("No Solution Exists")
```

Practical No: 5

Implement Local Beam Search for Traveling Salesman Problem.

Title

Implementation of Local Beam Search for Traveling Salesman Problem (TSP)

Objectives

- To understand the concept of **Local Beam Search algorithm** in Artificial Intelligence.
- To implement Local Beam Search for solving the Traveling Salesman Problem.
- To find an optimal or near-optimal path that visits all cities exactly once.
- To minimize the total travel distance between cities.
- To analyze the performance of heuristic search techniques for optimization problems.

Problem Statement

Implement the **Local Beam Search algorithm for the Traveling Salesman Problem (TSP).**

Given a set of cities and distances between them, find the shortest possible route that:

- Starts from a given city.
- Visits every city exactly once.
- Returns to the starting city.

The algorithm should maintain **K possible solutions (states)** at each step and select the best K states for further exploration.

Software Requirements:

- **Programming Language:** Python 3.x

Platform / IDE:

- Visual Studio Code
- PyCharm
- Jupyter Notebook

Operating System:

- Windows / Linux / macOS

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Traveling Salesman Problem (TSP)

The Traveling Salesman Problem is a famous optimization problem where a salesman has to find the shortest route to visit multiple cities exactly once and return to the starting city.

Local Beam Search

Local Beam Search is a heuristic search algorithm that keeps multiple candidate solutions instead of a single solution.

It works as follows:

1. Generate multiple initial states.
2. Evaluate their fitness values.
3. Select the best K states.
4. Generate successors of selected states.
5. Repeat until the optimal solution is found.

Important Terms

State: A possible route arrangement of cities.

Fitness Function:

Total distance of the route.

Lower distance = Better solution.

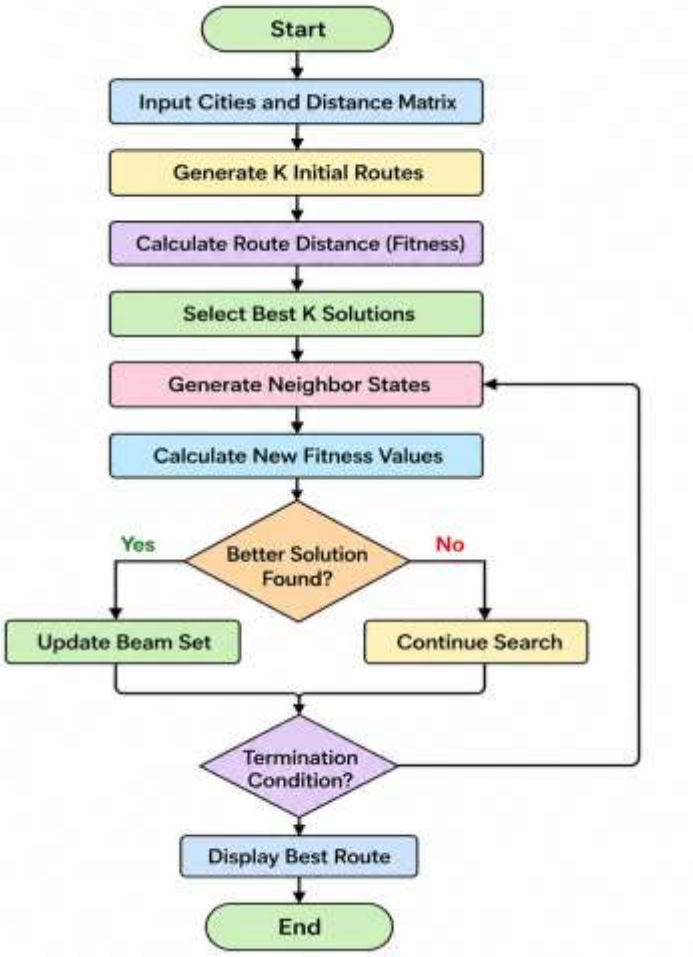
Beam Width (K):

Number of best states maintained during search.

Algorithm

1. Start the program.
2. Input cities and distance matrix.
3. Generate K random initial routes.
4. Calculate the distance of each route.
5. Select the best K routes.
6. Generate neighboring routes by swapping cities.
7. Evaluate new routes.
8. Keep only the best K routes.
9. Repeat until stopping condition is reached.
10. Display the shortest route and minimum distance.

FlowChart



Conclusion

The Local Beam Search algorithm was successfully implemented for solving the Traveling Salesman Problem. The algorithm maintains multiple candidate solutions and improves them iteratively by selecting the best routes. It provides an efficient heuristic approach for finding near-optimal solutions for large optimization problems.

Practical Questions

1. Implement Hill Climbing algorithm for solving the Traveling Salesman Problem.
2. Compare Local Beam Search and Genetic Algorithm for TSP optimization.
3. Implement Simulated Annealing algorithm for route optimization.
4. Apply Local Beam Search for solving the 8-Puzzle problem.
5. Implement Local Search techniques for job scheduling optimization.

Code:

```
import random
distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
cities = [0, 1, 2, 3]
def calculate_distance(route):
    total = 0
    for i in range(len(route)-1):
        total += distance[route[i]][route[i+1]]
    total += distance[route[-1]][route[0]]
    return total
def generate_neighbors(route):
    neighbors = []
    for i in range(len(route)):
        for j in range(i+1, len(route)):
            new_route = route.copy()
            new_route[i], new_route[j] = (
                new_route[j],
                new_route[i]
            )
```

```
            neighbors.append(new_route)
    return neighbors
def local_beam_search(k, iterations):
    beam = []
    for i in range(k):
        route = cities.copy()
        random.shuffle(route)
        beam.append(route)
    for i in range(iterations):
        candidates = []
        for route in beam:
            candidates.extend(
                generate_neighbors(route)
            )
        candidates.sort(
            key=calculate_distance
        )
        beam = candidates[:k]
    best_route = beam[0]
    return best_route, calculate_distance(best_route)
route, cost = local_beam_search(3,10)
print("Best Route:", route)
print("Minimum Distance:", cost)
```

Practical No: 6

Implementation of Breadth First Search (BFS) for Solving 8-Puzzle Problem

Title

Implementation of Breadth First Search (BFS) for Solving 8-Puzzle Problem

Objectives

- To understand the concept of **Breadth First Search (BFS)** in Artificial Intelligence.
- To represent a planning problem using **state space representation**.
- To implement BFS algorithm to find the shortest path from initial state to goal state.
- To solve the 8-Puzzle problem using an uninformed search technique.
- To analyze the performance of BFS in solving planning problems.

Problem Statement

Implement **Breadth First Search (BFS)** to solve the **8-Puzzle problem**.
The 8-Puzzle consists of a 3×3 board containing eight numbered tiles and one blank space. The objective is to move tiles by sliding them into the blank space until the goal state is reached.

Rules:

1. A tile can move only into the empty space.
2. Only one tile can be moved at a time.
3. The final arrangement should match the goal state.

Initial State:

1 2 3

4 0 6

7 5 8

Goal State:

1 2 3

4 5 6

7 8 0

Where:

0 represents the blank space.

Software Requirements:

- **Programming Language:** Python 3.x

Platform / IDE:

- Visual Studio Code
- PyCharm
- Jupyter Notebook

Operating System:

- Windows / Linux / macOS

Hardware Requirements:

Processor: Intel i3 or above

RAM: Minimum 4 GB

Storage: Minimum 500 MB free space

Theory

Breadth First Search (BFS)

Breadth First Search is an uninformed search algorithm that explores the state space level by level.

It uses a **Queue (FIFO)** data structure.

BFS always finds the shortest path if:

- Each step has equal cost.
- The solution exists.

State Space Representation

A planning problem can be represented using:

1. Initial State

The starting configuration of the problem.

2. Operators

Possible actions that change the state.

For 8-Puzzle:

- Move blank space Up
- Move blank space Down
- Move blank space Left
- Move blank space Right

3. Goal State
The desired final configuration.
1 2 3
4 5 6
7 8 0

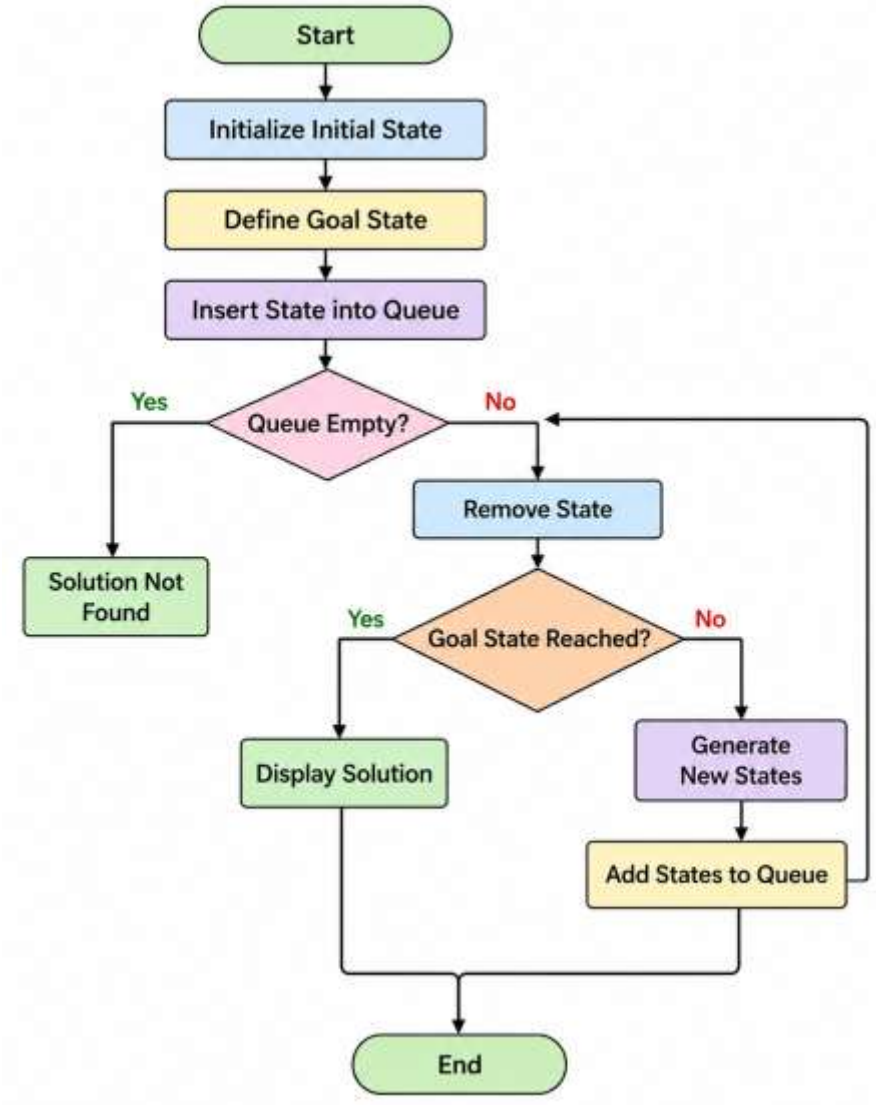
BFS Working

1. Insert initial state into queue.
2. Remove the first state from queue.
3. Generate all possible child states.
4. Check whether goal state is reached.
5. If not, add new states into queue.
6. Continue until solution is found.

Algorithm

1. Start the program.
2. Define initial state and goal state.
3. Create an empty queue.
4. Insert initial state into the queue.
5. Remove a state from the queue.
6. Check whether the current state is the goal state.
7. If goal state is reached, display solution path.
8. Otherwise generate possible moves.
9. Add new states into the queue.
10. Repeat until the solution is found.
11. Stop.

Flowchart



Conclusion

The 8-Puzzle problem was successfully solved using Breadth First Search. The problem was represented as a state-space search problem with initial state, operators, and goal state. BFS successfully finds the shortest solution path by exploring states level by level.

Practical Questions

1. Implement BFS algorithm for Robot Path Planning.
2. Solve the Blocks World problem using BFS search.
3. Implement Depth First Search (DFS) for solving the 8-Puzzle problem.
4. Compare BFS and DFS algorithms for state space search problems.
5. Implement BFS for finding the shortest path in a maze navigation problem.

Code;

```
from collections import deque

def get_neighbors(state):

    neighbors = []

    index = state.index(0)

    row = index // 3
    col = index % 3


    moves = [
        (-1,0),
        (1,0),
        (0,-1),
        (0,1)
    ]

    for move in moves:

        new_row = row + move[0]
        new_col = col + move[1]


        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_index = new_row * 3 + new_col

            new_state = list(state)

            new_state[index], new_state[new_index] = (
                new_state[new_index],
                new_state[index]
            )

            neighbors.append(tuple(new_state))


    return neighbors
```

```
def bfs(start, goal):

    queue = deque()

    queue.append((start, []))

    visited = set()

    visited.add(start)


    while queue:

        current, path = queue.popleft()


        if current == goal:
            return path + [current]


        for neighbour in get_neighbors(current):

            if neighbour not in visited:

                visited.add(neighbour)

                queue.append(
                    (neighbour, path+[current])
                )

    return None


initial_state = (
    1,2,3,
    4,0,6,
    7,5,8
)


goal_state = (
    1,2,3,
    4,5,6,
```

```
    7,8,0
)


solution = bfs(initial_state, goal_state)


print("Solution Path:")

for state in solution:
    print(state)
```

Practical No: 7

Use DFS for problem solving. Examples can be water jug problem , Missionaries and Cannibals ,maze solving, blocks world etc.

Title

Implementation of Depth First Search (DFS) for Solving the Water Jug Problem

Objectives

- To understand the concept of **Depth First Search (DFS)** in Artificial Intelligence.
- To represent the Water Jug Problem as a **state space search** problem.
- To implement the DFS algorithm for finding a solution.
- To explore states using a depth-first approach.
- To analyze the performance of DFS in solving planning problems.

Problem Statement

Implement the **Depth First Search (DFS)** algorithm to solve the **Water Jug Problem**. Given two water jugs with capacities of **4 liters** and **3 liters**, determine the sequence of operations required to obtain exactly **2 liters** of water in the 4-liter jug.

Allowed Operations:

1. Fill either jug completely.
2. Empty either jug completely.
3. Pour water from one jug to the other until one jug is empty or the other is full.

Software Requirements

- **Programming Language:** Python 3.x

Platform / IDE

- Visual Studio Code (VS Code)
- PyCharm
- Jupyter Notebook

Operating System

- Windows / Linux / macOS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Depth First Search (DFS)

Depth First Search (DFS) is an **uninformed search algorithm** that explores one branch of the search tree as deeply as possible before backtracking.

DFS uses a **Stack (LIFO)** data structure or recursion.

Characteristics:

- Explores deep into the search tree before exploring siblings.
- Uses less memory than BFS.
- May not always find the shortest path.

Water Jug Problem

The Water Jug Problem is a classic **state space search** problem.

State Representation

A state is represented as: (J1,J2)

Where:

- J1J_1J1 = Amount of water in the 4-liter jug.
- J2J_2J2 = Amount of water in the 3-liter jug.

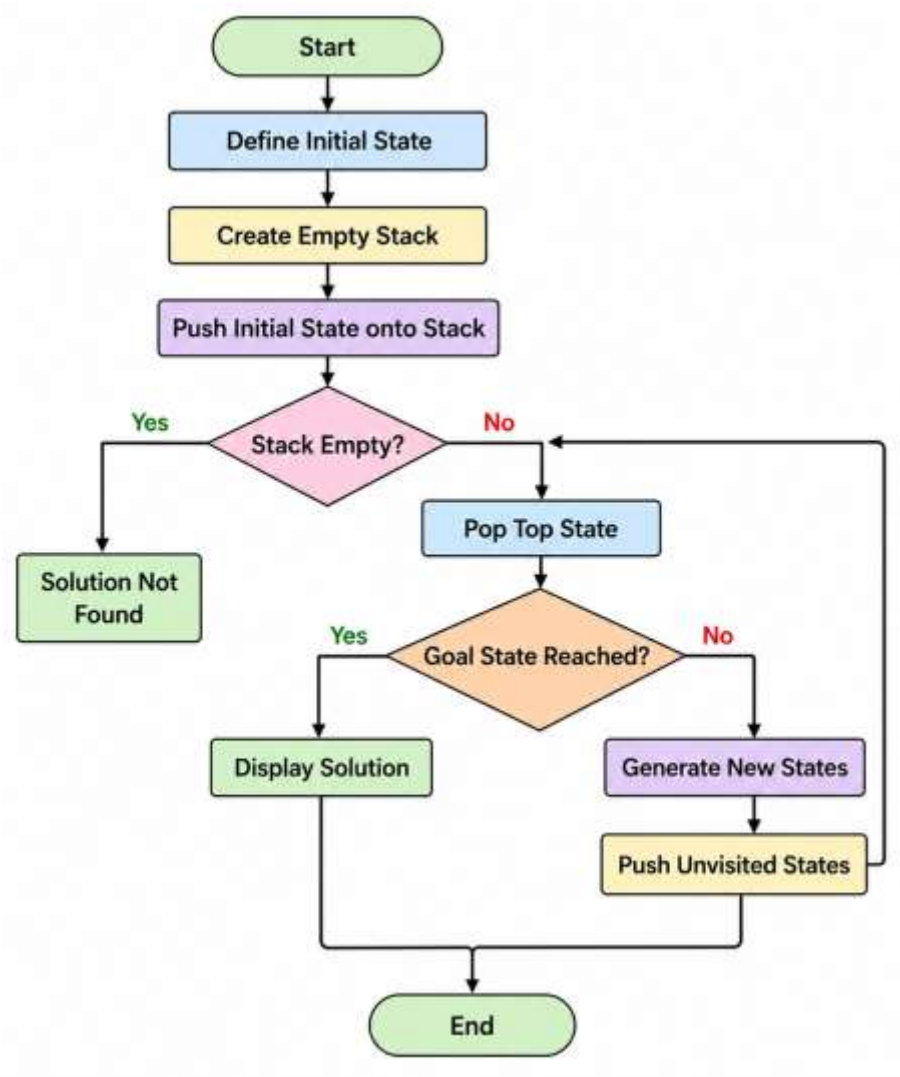
Initial State : (0,0)

Goal State : (2,0)

Algorithm:

1. Start the program.
2. Define the initial state and goal state.
3. Create an empty stack.
4. Push the initial state onto the stack.
5. Pop the top state from the stack.
6. Check whether the current state is the goal state.
7. If the goal is reached, display the solution path.
8. Otherwise, generate all possible valid next states.
9. Push unvisited states onto the stack.
10. Repeat Steps 5–9 until the goal is reached or the stack becomes empty.
11. Stop.

Flowchart



Mathematical Model

State Representation

S=(J1,J2)S=(J_1,J_2)S=(J1,J2)

Where:

- J1J_1J1 = Water in 4-liter jug
- J2J_2J2 = Water in 3-liter jug

Initial State

S0=(0,0)S_0=(0,0)S0=(0,0)

Goal State

Sg=(2,0)S_g=(2,0)Sg=(2,0)

State Transition Function

T(S)={Fill,Empty,Pour}T(S)=\{Fill, Empty, Pour\}T(S)={Fill,Empty,Pour}

Time Complexity

O(V+E)O(V+E)O(V+E)

Where:

- VVV = Number of states
- EEE = Number of transitions between states

Space Complexity

O(V)O(V)O(V)

Conclusion

The Water Jug Problem was successfully solved using the **Depth First Search (DFS)** algorithm. The problem was modeled as a state space search, and DFS explored the states by going as deep as possible before backtracking. This practical demonstrates the application of DFS in solving planning and state-space search problems in Artificial Intelligence.

Practical Questions

1. Implement the Missionaries and Cannibals problem using DFS.
2. Solve a maze navigation problem using Depth First Search.
3. Implement DFS for the Blocks World problem.
4. Compare Breadth First Search (BFS) and Depth First Search (DFS) for state space search problems.
5. Implement DFS to solve the Robot Path Planning problem.

Code:

```
def water_jug_dfs():
    stack = [((0, 0), [])]
    visited = set()

    while stack:
        (jug1, jug2), path = stack.pop()
```

```
        if (jug1, jug2) in visited:
            continue

        visited.add((jug1, jug2))
        path = path + [(jug1, jug2)]

              if jug1 == 2:
            return path

        next_states = [
            (4, jug2),
            (jug1, 3),
            (0, jug2),
            (jug1, 0),
            (jug1 - min(jug1, 3-jug2),
             jug2 + min(jug1, 3-jug2)),
            (jug1 + min(jug2, 4-jug1),
             jug2 - min(jug2, 4-jug1))
        ]

        for state in next_states:
            if state not in visited:
                stack.append((state, path))

    return None


solution = water_jug_dfs()

print("Solution Path:")
for step in solution:
    print(step)
```

Practical No: 8

To implement Backtracking search to solve the 8-Queens constraint satisfaction problem.

Title

Implementation of Backtracking Search to Solve the 8-Queens Constraint Satisfaction Problem

Objectives

- To understand the concept of **Constraint Satisfaction Problems (CSP)** in Artificial Intelligence.
- To implement the **Backtracking Search** algorithm for solving the 8-Queens problem.
- To place eight queens on a chessboard such that no two queens attack each other.
- To apply constraint checking while searching for a valid solution.
- To analyze the efficiency of the Backtracking algorithm in solving CSPs.

Problem Statement

Implement the **Backtracking Search** algorithm to solve the **8-Queens Constraint Satisfaction Problem (CSP)**.

The objective is to place **8 queens on an 8×8 chessboard** such that:

- No two queens are placed in the same row.
- No two queens are placed in the same column.
- No two queens are placed on the same diagonal.

The program should generate one valid arrangement of queens using the Backtracking technique.

Software Requirements

- **Programming Language:** Python 3.x

Platform / IDE

- Visual Studio Code (VS Code)
- PyCharm
- Jupyter Notebook

Operating System

- Windows / Linux / macOS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Constraint Satisfaction Problem (CSP)

A **Constraint Satisfaction Problem (CSP)** consists of:

- **Variables**
- **Domains**
- **Constraints**

Variables

Each row of the chessboard represents a variable where one queen must be placed.

Domain

Each queen can be placed in any column (0–7).

Constraints

A queen cannot attack another queen.

Therefore:

- Only one queen per row.
- Only one queen per column.
- No two queens on the same diagonal.

Backtracking Search

Backtracking is a recursive search technique that:

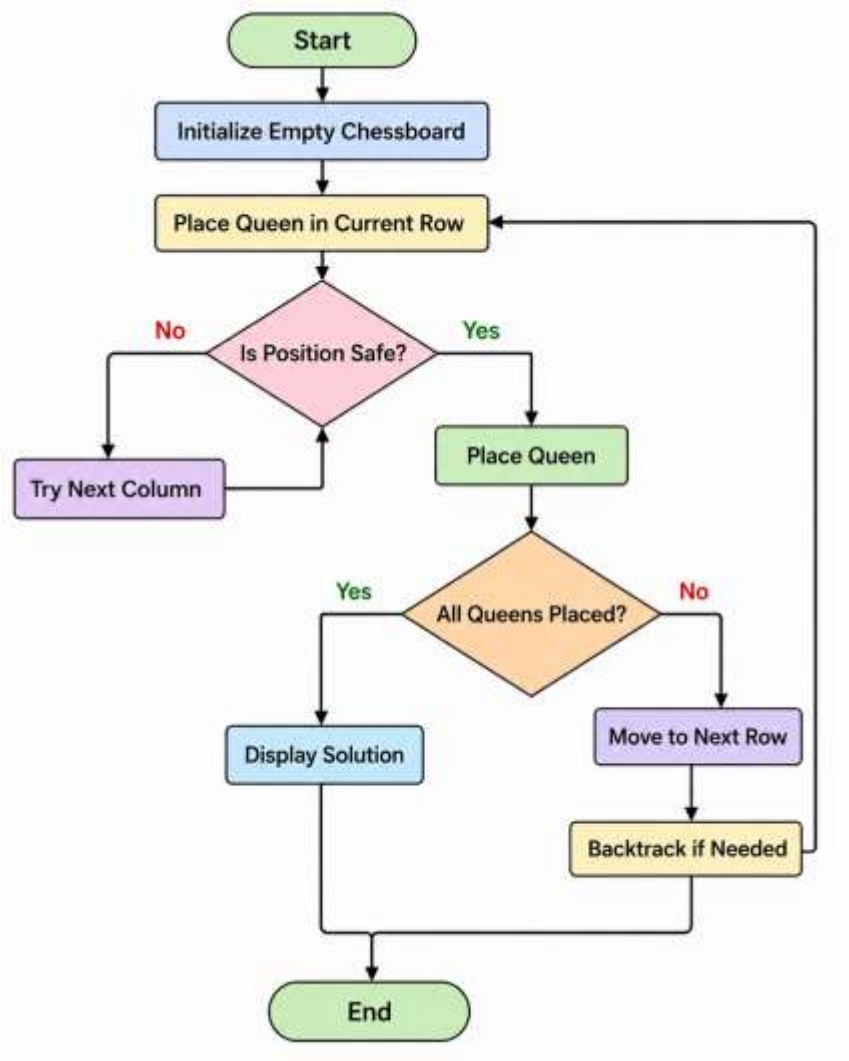
1. Places a queen in a safe position.
2. Moves to the next row.
3. If no safe position exists, it backtracks to the previous row.
4. Continues until all eight queens are placed.

Backtracking avoids unnecessary searches by rejecting invalid partial solutions earl

Algorithm

1. Start the program.
2. Create an empty 8×8 chessboard.
3. Begin placing queens from the first row.
4. Check whether the current position is safe.
5. If safe, place the queen.
6. Move to the next row.
7. If all rows are completed, display the solution.
8. If no safe position exists, remove the previously placed queen (Backtrack).
9. Repeat until a valid arrangement is found.
10. Stop

Flowchart



Conclusion

The 8-Queens Constraint Satisfaction Problem was successfully solved using the Backtracking Search algorithm. The algorithm efficiently places queens while checking row, column, and diagonal constraints, and backtracks whenever an invalid placement is encountered. This practical demonstrates the application of Backtracking in solving Constraint Satisfaction Problems in Artificial Intelligence.

Practical Questions

1. Implement the N-Queens Problem for any value of NNN using Backtracking.
2. Compare Backtracking and Branch and Bound approaches for solving the N-Queens problem.
3. Implement the Sudoku Solver using Backtracking Search.
4. Solve the Graph Coloring Problem using Backtracking.
5. Explain how Constraint Satisfaction Problems (CSPs) differ from general search problems with suitable examples.

Code:

```
N = 8

def print_board(board):
    for row in board:
        print(" ".join("Q" if cell else "." for cell in row))

def is_safe(board, row, col):

    for i in range(col):
        if board[row][i]:
            return False

     i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

      i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1
```

```
    return True


def solve(board, col):

    if col >= N:
        return True

    for row in range(N):

        if is_safe(board, row, col):

            board[row][col] = 1

            if solve(board, col + 1):
                return True

            board[row][col] = 0

    return False


board = [[0 for _ in range(N)] for _ in range(N)]

if solve(board, 0):
    print("Solution:")
    print_board(board)
else:
    print("No Solution Exists")
```

Practical No: 9

To develop a simple chatbot using predefined rules and pattern matching.

Title
Development of a Simple Chatbot using Predefined Rules and Pattern Matching

Objectives

1. To understand the basic concepts of Artificial Intelligence and chatbots.
2. To develop a simple chatbot using predefined rules and pattern matching.
3. To simulate human-like conversation by responding to user queries.
4. To understand how rule-based systems work in Natural Language Processing (NLP).
5. To analyze the advantages and limitations of rule-based chatbots.

Problem Statement
Develop a **simple chatbot** using **predefined rules and pattern matching**. The chatbot should accept user input, identify keywords or patterns, and generate appropriate responses from a predefined set of rules.

Example:

- User: Hello
- Chatbot: Hello! How can I help you today?

The chatbot should respond to common greetings, questions, and exit commands.

Software Requirements

- **Programming Language:** Python 3.x

Platform / IDE

- Visual Studio Code (VS Code)
- PyCharm
- Jupyter Notebook

Operating System

- Windows / Linux / macOS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Chatbot

A **chatbot** is a software application that interacts with users through text or voice. It is designed to answer questions, provide information, or perform specific tasks automatically.

Examples:

- Customer Support Bot
- Banking Chatbot
- Healthcare Assistant
- College Information Chatbot

Rule-Based Chatbot
A **rule-based chatbot** works by following predefined rules and matching user input with stored patterns.
It does not learn from conversations but provides responses based on fixed conditions.

User Input	Chatbot Response
Hello	Hello! How can I help you?
Hi	Hi! Nice to meet you.
How are you?	I'm fine. Thank you!
Bye	Goodbye! Have a nice day.

Pattern Matching

Pattern matching identifies keywords or phrases in the user's input and returns the corresponding response.

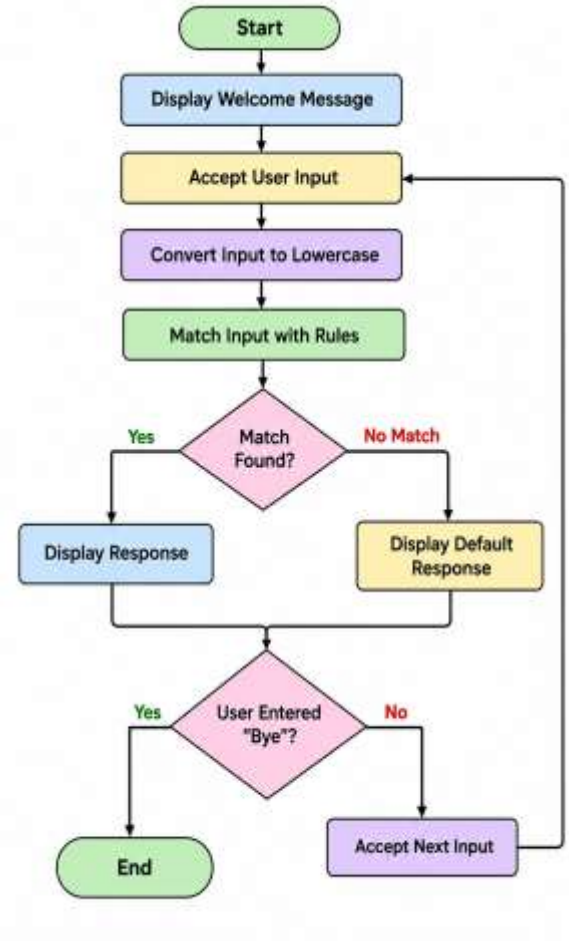
Example:
If input contains "hello"
    Response = "Hello! Welcome."

If input contains "bye"
    Response = "Goodbye!"

Algorithm:

1. Start the program.
2. Display a welcome message.
3. Accept user input.
4. Convert the input to lowercase.
5. Compare the input with predefined rules.
6. If a matching pattern is found, display the corresponding response.
7. If no pattern matches, display a default response.
8. Repeat until the user enters **"bye"** or **"exit"**.
9. Stop the program.

Flowchart



Conclusion

A simple chatbot was successfully developed using predefined rules and pattern matching. The chatbot accepts user input, compares it with predefined patterns, and provides appropriate responses. This practical demonstrates the basic concepts of Artificial Intelligence, Natural Language Processing (NLP), and rule-based systems. Although it cannot learn from conversations, it serves as a foundation for developing more advanced AI-powered chatbots.

Practical Questions

1. Develop a chatbot using Natural Language Processing (NLP) techniques.
2. Compare rule-based chatbots and AI-based chatbots.
3. Implement a college enquiry chatbot using predefined rules.
4. Develop a chatbot using Python's NLTK library for text processing.
5. Explain the advantages and limitations of pattern matching in chatbot development.

Code:

```
def chatbot():
    print("Chatbot: Hello! I am your AI Chatbot.")
    print("Type 'bye' to exit.\n")

    while True:
        user = input("You: ").lower()

        if user in ["hello", "hi", "hey"]:
            print("Chatbot: Hello! How can I help you?")

        elif "how are you" in user:
            print("Chatbot: I am fine. Thank you!")

        elif "your name" in user:
            print("Chatbot: I am a simple AI Chatbot.")

        elif "course" in user:
            print("Chatbot: We offer courses in AI, Data Science, Python, and Machine
Learning.")

        elif "college" in user:
            print("Chatbot: Welcome to the Engineering College Information System.")

        elif user in ["bye", "exit"]:
            print("Chatbot: Goodbye! Have a great day.")
            break

        else:
            print("Chatbot: Sorry, I don't understand your question.")

chatbot()
```

Practical No: 10

To develop a game-playing agent using the Minimax algorithm for optimal decision-making.

Title

Development of a Game-Playing Agent using the Minimax Algorithm for Optimal Decision-Making

Objectives

1. To understand the concept of **Adversarial Search** in Artificial Intelligence.
2. To implement the **Minimax algorithm** for optimal decision-making in two-player games.
3. To develop a simple game-playing agent that selects the best possible move.
4. To analyze how Minimax predicts the opponent's moves.
5. To understand the role of game trees in AI-based game playing.

Problem Statement

Develop a **game-playing agent** using the **Minimax algorithm** for a two-player game. The agent should evaluate all possible moves, assume that both players play optimally, and select the move that maximizes its chances of winning while minimizing the opponent's advantage.

Example:

The game-playing agent can be implemented for **Tic-Tac-Toe**.

The AI should always choose the optimal move using the Minimax algorithm.

Software Requirements

- **Programming Language:** Python 3.x

Platform / IDE

- Visual Studio Code (VS Code)
- PyCharm
- Jupyter Notebook

Operating System

- Windows / Linux / macOS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Game-Playing in Artificial Intelligence

Game-playing is an application of Artificial Intelligence where an agent makes decisions in a competitive environment. The objective is to choose the best possible move based on the current game state

Examples:

- Tic-Tac-Toe
- Chess
- Checkers
- Connect Four

Minimax Algorithm
The **Minimax algorithm** is an adversarial search algorithm used in two-player games.

It assumes:

- **MAX Player (AI):** Tries to maximize the score.
- **MIN Player (Opponent):** Tries to minimize the score.

The algorithm explores all possible game states and selects the move that provides the best outcome for the AI.

Working of Minimax

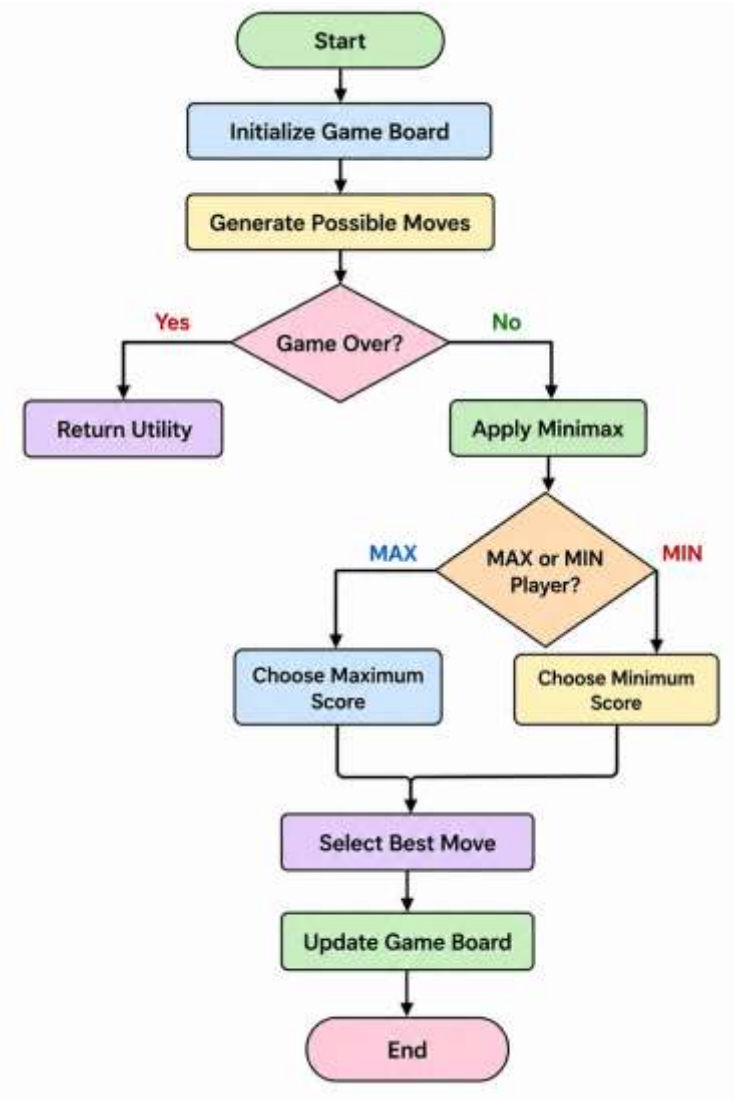
1. Generate all possible moves.
2. Evaluate terminal game states.
3. Assign utility values to terminal states.
4. Propagate values upward through the game tree.
5. Select the move with the highest utility for the MAX player.

Algorithm:

1. Start the program.
2. Initialize the game board.
3. Check whether the game is over.
4. If terminal state, return the utility value.
5. If it is the AI's turn (MAX), evaluate all possible moves and choose the maximum value.
6. If it is the opponent's turn (MIN), evaluate all possible moves and choose the minimum value.
7. Repeat recursively until all possible moves are evaluated.
8. Return the optimal move.
9. Display the updated game board.

10. Stop when the game ends.

Flowchart



Mathematical Model

Players

- MAX = Artificial Intelligence
- MIN = Opponent

Utility Function

U(s)={+1,if AI wins0,if Draw−1,if Opponent winsU(s)= \begin{cases} +1, & \text{if AI wins}\\ 0, & \text{if Draw}\\ -1, & \text{if Opponent wins} \end{cases}U(s)= ⎩⎨⎧+1,0,−1,if AI winsif Drawif Opponent wins

Minimax Functions

For MAX Player:

V(s)=max⁡(V(s1),V(s2),...,V(sn))V(s)=\max(V(s_1),V(s_2),...,V(s_n))V(s)=max(V(s1),V(s2),...,V(sn))

For MIN Player:

V(s)=min⁡(V(s1),V(s2),...,V(sn))V(s)=\min(V(s_1),V(s_2),...,V(s_n))V(s)=min(V(s1),V(s2),...,V(sn))

Time Complexity

O(bd)O(b^d)O(bd)

Where:

- bbb = Branching factor
- ddd = Depth of the game tree

Space Complexity

O(b×d)O(b \times d)O(b×d)

Conclusion

A game-playing agent was successfully developed using the Minimax algorithm. The algorithm evaluates all possible future game states and selects the optimal move by maximizing the AI's advantage while minimizing the opponent's advantage. This practical demonstrates the application of adversarial search and decision-making techniques in Artificial Intelligence.

Practical Questions

1. Implement the Minimax algorithm for a complete Tic-Tac-Toe game.
2. Compare the Minimax algorithm and Alpha-Beta Pruning in terms of efficiency.
3. Develop a Connect Four game-playing agent using Minimax.
4. Explain the role of utility functions in adversarial search algorithms.
5. Modify the Minimax algorithm to include Alpha-Beta Pruning and compare the execution time.

Code:

```
def minimax(depth, is_maximizing):
    if depth == 0:
        return 0

    if is_maximizing:
        best = -1000

        for i in range(2):
            score = minimax(depth - 1, False)
            best = max(best, score)

        return best

    else:
        best = 1000

        for i in range(2):
            score = minimax(depth - 1, True)
            best = min(best, score)

        return best


depth = 3

result = minimax(depth, True)

print("Optimal Utility Value:", result)
```

Practical No: 11

To optimize adversarial search by implementing Alpha-Beta pruning and compare its efficiency with Minimax.

Title

Implementation of Alpha-Beta Pruning to Optimize Adversarial Search and Compare its Efficiency with Minimax

Objectives

1. To understand the concept of **Adversarial Search** in Artificial Intelligence.
2. To implement the **Minimax algorithm** and **Alpha-Beta Pruning**.
3. To optimize game-tree search by eliminating unnecessary branches.
4. To compare the performance of Alpha-Beta Pruning with the Minimax algorithm.
5. To analyze the reduction in search time and number of nodes evaluated.

Problem Statement

Implement the **Alpha-Beta Pruning algorithm** to optimize adversarial search in a two-player game. Compare its performance with the **Minimax algorithm** by evaluating the number of nodes explored and the execution efficiency.

The implementation should:

- Generate a game tree.
- Apply the Minimax algorithm to determine the optimal move.
- Apply Alpha-Beta Pruning on the same game tree.
- Compare the results based on execution time and nodes evaluated.

Software Requirements

- **Programming Language:** Python 3.x

Platform / IDE

- Visual Studio Code (VS Code)
- PyCharm
- Jupyter Notebook

Operating System

- Windows / Linux / macOS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

Theory

Adversarial Search

Adversarial search is used in competitive environments where one player's gain is another player's loss.

Examples:

- Chess
- Tic-Tac-Toe
- Checkers
- Connect Four

The search process uses a **game tree** to evaluate all possible moves.

Minimax Algorithm

The **Minimax algorithm** explores every possible move in the game tree.

- **MAX Player:** Maximizes the utility value.
- **MIN Player:** Minimizes the utility value.

Although Minimax guarantees the optimal move, it evaluates every node, making it computationally expensive.

Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization of the Minimax algorithm.

It avoids exploring branches that cannot influence the final decision.

Alpha (α)

Best value found so far for the MAX player.

Beta (β)

Best value found so far for the MIN player.

Pruning Condition

α≥β\alpha \geq \betaα≥β

When this condition becomes true, the remaining branches are pruned.

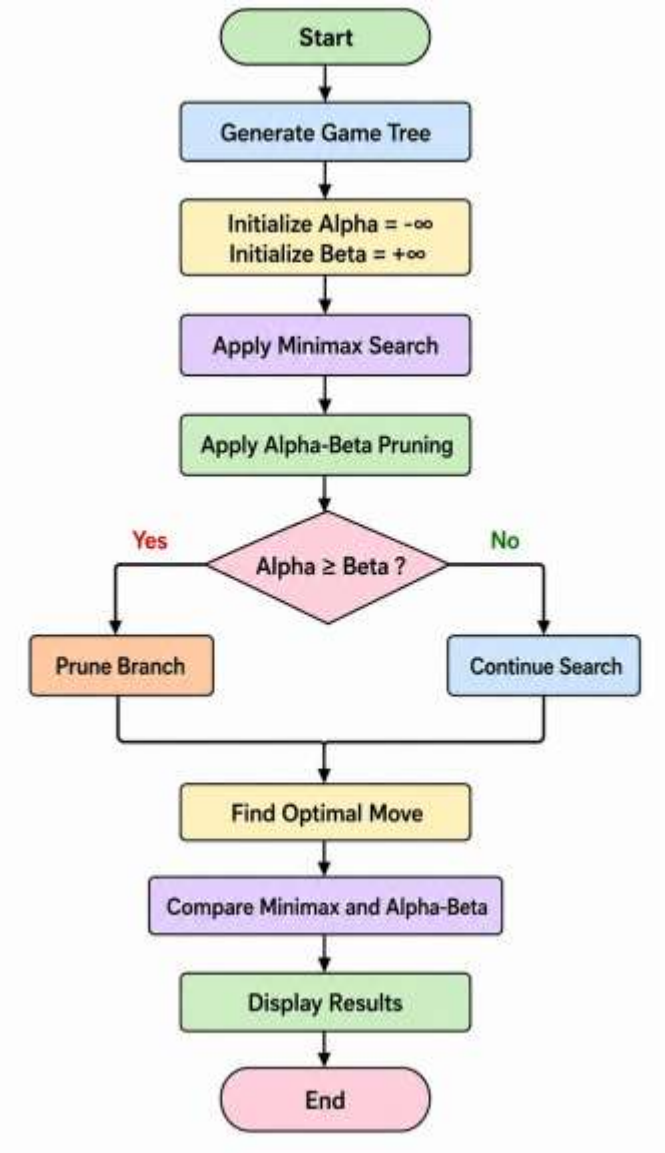
Advantages of Alpha-Beta Pruning

- Reduces the number of nodes evaluated.
- Produces the same optimal result as Minimax.
- Improves execution speed.
- Efficient for large game trees.

Algorithm:

1. Start the program.
2. Create the game tree.
3. Initialize:
    - Alpha = −∞
    - Beta = +∞
4. Apply the Minimax algorithm to evaluate the game tree.
5. Apply Alpha-Beta Pruning.
6. Update Alpha and Beta values.
7. If Alpha ≥ Beta, prune the remaining branches.
8. Continue until the optimal move is found.
9. Compare the number of nodes evaluated by Minimax and Alpha-Beta Pruning.
10. Display the optimal move and comparison results.
11. Stop

Flowchart



Mathematical Model

Utility Function

$$U(s)=\{+1,\text{Win}0,\text{Draw}-1,\text{Loss}U(s)= \begin{cases} +1, & \text{Win}\\ 0, & \text{Draw}\\ -1, & \text{Loss} \end{cases}U(s)= \begin{cases} +1,0,-1,\text{WinDrawLoss} \end{cases}$$

MAX Function

MAX(n)=max⁡(V1,V2,…,Vn)MAX(n)=\max(V_1,V_2,\ldots,V_n)MAX(n)=max(V1,V2,…,Vn)

MIN Function

MIN(n)=min⁡(V1,V2,…,Vn)MIN(n)=\min(V_1,V_2,\ldots,V_n)MIN(n)=min(V1,V2,…,Vn)

Alpha-Beta Pruning Condition

α≥β\alpha \geq \betaα≥β

Time Complexity

Minimax

O(bd)O(b^d)O(bd)

Alpha-Beta Pruning (Best Case)

O(bd/2)O(b^{d/2})O(bd/2)

Where:

- bbb = Branching factor
- ddd = Depth of the game tree

Conclusion

The Alpha-Beta Pruning algorithm was successfully implemented to optimize adversarial search. Compared to the Minimax algorithm, Alpha-Beta Pruning evaluates fewer nodes while producing the same optimal result. This significantly improves search efficiency and reduces execution time, making it suitable for complex game-playing applications such as Chess and Tic-Tac-Toe.

Practical Questions

1. Implement the Minimax algorithm for a Tic-Tac-Toe game and compare it with Alpha-Beta Pruning.
2. Explain how Alpha-Beta Pruning reduces the number of nodes explored in a game tree.
3. Compare the time complexity of Minimax and Alpha-Beta Pruning with suitable examples.
4. Implement Alpha-Beta Pruning for a simple Chess game tree.
5. Explain the roles of Alpha (α) and Beta (β) values with an example.

Code:

```
def alpha_beta(depth, node_index, maximizing_player, values, alpha, beta):

    if depth == 3:
      return values[node_index]

  if maximizing_player:
    best = float('-inf')

    for i in range(2):
      value = alpha_beta(depth + 1,
                  node_index * 2 + i,
                  False,
                  values,
                  alpha,
                  beta)

      best = max(best, value)
      alpha = max(alpha, best)

      if alpha >= beta:
        break   # Pruning

    return best

  else:
    best = float('inf')

    for i in range(2):
      value = alpha_beta(depth + 1,
                  node_index * 2 + i,
                  True,
                  values,
                  alpha,
                  beta)

      best = min(best, value)
      beta = min(beta, best)
```

```
            if alpha >= beta:
                break

        return best


values = [3, 5, 6, 9, 1, 2, 0, -1]

result = alpha_beta(0, 0, True, values,
                    float('-inf'),
                    float('inf'))

print("Optimal Value:", result)
```